Insurance Charge Predictor

AI545 - W26 Project Submission

S M Ahsanul Kabir

May 2, 2026

Contents

1 Problem Statement **3**

2 Project Specification **3**
2.1 Project Structure . . . 3
2.2 How Project Works . . . 4
2.3 File-by-File Reference . . . 4
2.3.1 Configuration . . . 4
2.3.2 Python source (`src/`) . . . 5
2.3.3 Dev ergonomics . . . 5
2.3.4 Containerization & CI/CD . . . 5
2.3.5 Kubernetes (`k8s/`) . . . 6
2.3.6 Data & artifacts . . . 6

3 Core MLOps Components **6**
3.1 Data Versioning Pipeline (DVC) . . . 6
3.2 Experiment Tracking (MLflow + Dummy Registry) . . . 6
3.3 Deployment Pipeline (Docker) . . . 7
3.4 Orchestration (Kubernetes) . . . 7
3.5 CI/CD Pipeline (GitHub Actions) . . . 7
3.6 Observability (New Relic) . . . 8

4 Challenges Faced **9**

1 Problem Statement

This project addresses a supervised machine learning regression problem. It predicts a person's medical insurance charges from input attributes such as age, gender, BMI, number of children, smoking habit, and region. In this system, the data are split into training and test sets, multiple candidate regression models are trained, and the model with the highest R^2 is selected.

This is an end-to-end MLOps demo that trains a regression model on the classic Kaggle Medical Cost Personal Dataset, tracks experiments with MLflow, promotes a champion model via a lightweight JSON registry, and serves predictions through a Streamlit UI — packaged as a Docker image and deployable to Hugging Face (HF) Spaces or Kubernetes from the same CI pipeline.

2 Project Specification

- Website: Insurance Charge Predictor in Hugging Face
- GitHub: ai-545-project
- Dataset: Medical Cost Personal Datasets
- Stack: Python 3.11, Docker, DVC, GitHub Actions, minikube (locally)

2.1 Project Structure

```
Dockerfile                      # Container image definition
Makefile                        # macOS/Linux task runner
README.md
newrelic.ini                    # New Relic agent config
params.yml                      # Central project configuration
requirements.txt                # Pinned Python dependencies
.github/workflows
   ci-cd.yml                    # GitHub Actions workflow
.streamlit/
   config.toml                  # Streamlit theme
data/
   insurance.csv                # Raw dataset (1,338 rows)
   insurance.csv.dvc            # DVC pointer for the CSV
k8s/
   deployment.yaml              # Deployment: 2 replicas, probes, resource limits
   service.yaml                 # NodePort service on 30085 → 8501
   newrelic-secret.example.yaml # Secrets for New Relic
models/
   metrics.json                 # Challenger metrics + leaderboard
   model.pkl                    # Serialized sklearn pipeline (champion)
registry/
   champion.json                # Dummy registry with all models info
scripts/
   entrypoint.sh                # Container entrypoint
   make.ps1                     # Windows PowerShell equivalent of Makefile
src/
   __init__.py
```

```
   app.py                          # Streamlit frontend
   register.py                     # Challenger → Champion promotion
   train.py                        # Multi-model training + MLflow logging
   utils.py                        # Shared helpers
tests/
    test_model.py                  # Smoke tests
```

2.2 How Project Works

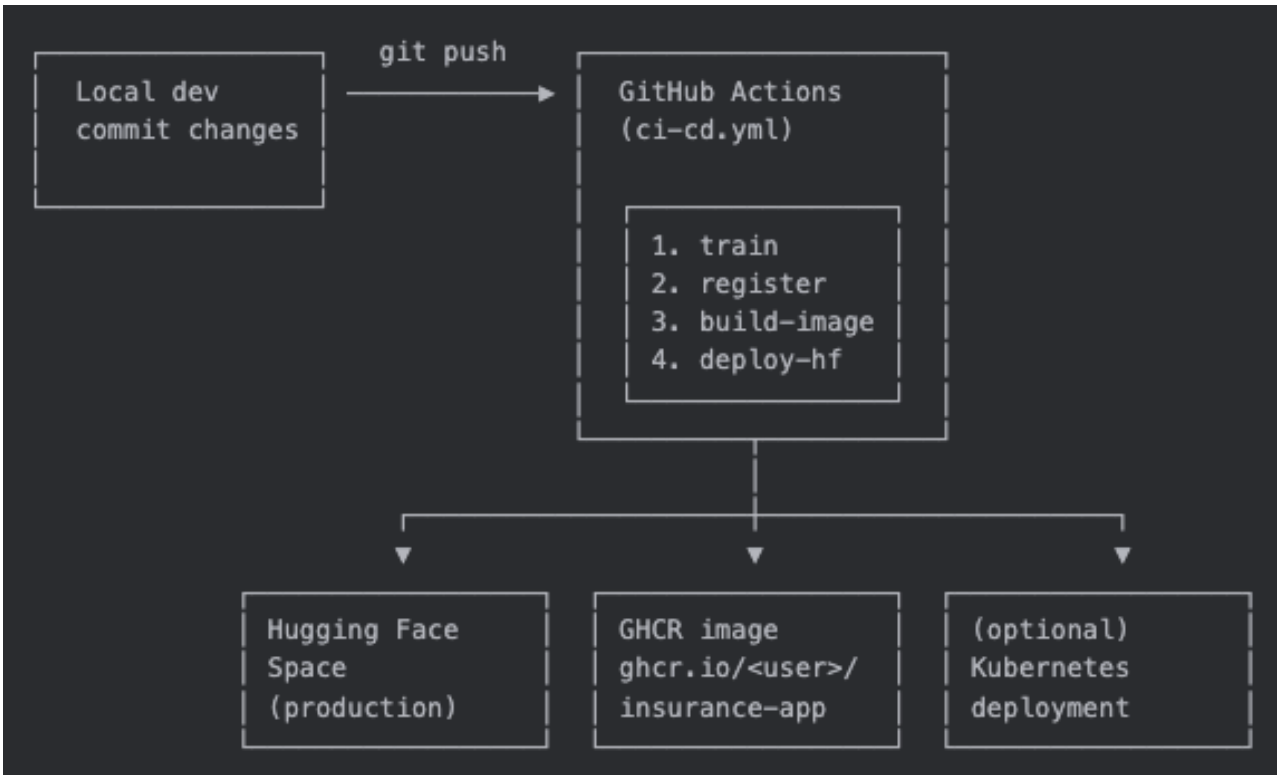

Figure 1: How the Project Works — Local to Prod

2.3 File-by-File Reference

2.3.1 Configuration

File	Purpose
`params.yml`	Defined dataset URL, feature ranges (age 18–64, BMI 15–55, etc.), target column, candidate models with hyperparameters, MLflow experiment name, output paths, app port. Both `train.py` and `app.py` read from this file.
`requirements.txt`	Pinned deps: pandas 2.2.3, numpy 1.26.4, scikit-learn 1.5.2, streamlit 1.39.0, mlflow 2.17.2, matplotlib, pyyaml, requests, joblib, pytest, newrelic 10.3.0.
`config.toml`	Defines streamlit theme
`newrelic.ini`	APM agent settings. License key + app name come from environment variables at runtime, no secret is committed.

2.3.2 Python source (`src/`)

File	Purpose
`utils.py`	Shared helpers: `project_root()`, `load_params()`, `download_data()` (cached fetch of insurance CSV), `load_dataset()`, `load_model()`/`save_model()`, and `demo_predict()` — a hand-coded formula used as a fallback when no `.pkl` is available.
`train.py`	Trains every candidate in `params.yml`, evaluates with R^2/MAE/RMSE, logs each as a nested MLflow run, picks the challenger by `selection_metric`, writes `model.pkl` and `metrics.json`. We used R^2 for this experiment.
`register.py`	Reads `metrics.json`, compares challenger vs. the existing `champion.json`, promotes only if better. Writes champion with a UTC timestamp and a `version`.
`app.py`	Streamlit UI. Three tabs, caches the dataset and the model via `st.cache_data` / `st.cache_resource`, records custom events to New Relic.
`__init__.py`	Marks `src/` as a package so `python -m src.train` works.

2.3.3 Dev ergonomics

File	Purpose
`Makefile`	All dev shortcuts: `venv`, `install`, `train`, `test`, `app`, `mlflow-ui`, `docker-*`, `k8s-*`, `fly-*`.
`scripts/make.ps1`	Windows PowerShell equivalent. Same targets, same semantics.
`scripts/entrypoint.sh`	Container entrypoint.

2.3.4 Containerization & CI/CD

File	Purpose
`Dockerfile`	`python:3.11-slim` base. Installs dependencies, copies app + model + config, exposes 8501, sets `HOME=/tmp`, adds a health-check, runs `entrypoint.sh`.
`ci-cd.yml`	Four sequential GitHub Actions jobs: `train` → `register` → `build-image` → `deploy-hf`. Artifacts flow between jobs via `upload-artifact` and `download-artifact`.

2.3.5 Kubernetes (k8s/)

File	Purpose
deployment.yaml	2 replicas, pulls image from GHCR, mounts NEW_RELIC_LICENSE_KEY from a Secret (optional), readiness + liveness probes on /_stcore/health, resources: 100m–1000m CPU, 256Mi–1Gi RAM.
service.yaml	NodePort service exposing port 8501 on node port 30085.
newrelic-secret	.example.yaml template for the New Relic Secret.

2.3.6 Data & artifacts

Path	Purpose
data/insurance.csv	Raw dataset (1,338 rows, 7 columns).
data/insurance.csv.dvc	DVC pointer tracking the hash of the CSV.
models/model.pkl	Serialized challenger sklearn.pipeline.Pipeline.
models/metrics.json	Challenger R^2/MAE/RMSE + full model metrics.
registry/champion.json	Dummy registry
tests/test_model.py	Smoke tests.

3 Core MLOps Components

3.1 Data Versioning Pipeline (DVC)

The repo is **DVC-initialized**: data/insurance.csv.dvc is the content-hash pointer that git tracks instead of the CSV itself. The intent is:

- The actual insurance.csv stays out of git (large binary, churn-prone).
- Its hash is committed, so a checkout of any past commit maps deterministically to the exact dataset that was used.
- dvc pull would fetch the file from a configured remote (S3, GCS, Azure Blob, or an SSH endpoint).

3.2 Experiment Tracking (MLflow + Dummy Registry)

Two distinct layers handle "which experiments ran" vs. "which model is deployed":

MLflow (experiment tracking): train.py uses a local file-store backend (./mlruns) and logs, for every candidate in params.yml:

- Parameters: model type, all hyperparameters, split config.
- Metrics: R^2, MAE, RMSE.
- Artifacts: the fitted sklearn.Pipeline under model/.
- Tags: challenger_name, challenger_type on the parent run.

Runs are grouped under a parent `training_session` run so candidates can be compared in the MLflow UI.

Dummy registry for promotion: Instead of using MLflow's Model Registry, which needs a real tracking server, this project uses a single file: `registry/champion.json`. `register.py` is the gatekeeper. It reads `metrics.json`, compares to the existing champion, and only overwrites the champion JSON if the new model wins on the configured metric.

3.3 Deployment Pipeline (Docker)

Terminology note: this project exposes a Streamlit **web UI**, not a REST API. There is no `/predict` HTTP endpoint you can `curl`. The container bundles the trained model with a web frontend.

Dockerfile strategy:

1. Start from `python:3.11-slim` for a small base.
2. Install `curl`, then wipe apt cache.
3. Copy `requirements.txt` first so the pip layer caches across builds.
4. Copy app code + model + registry **after** dependencies, so changing `app.py` doesn't invalidate the pip cache.
5. Set the Hugging Face environment variables.
6. `HEALTHCHECK` against `/_stcore/health` → Streamlit's built-in health endpoint, also used by the Kubernetes probes.
7. `CMD ["/entrypoint.sh"]` → one entrypoint, two modes (NR-wrapped or plain Streamlit) depending on whether the license key is present.

Image tags. CI produces one tags on every push to `main` for deployment:

- `ghcr.io/<owner>/insurance-app:latest`

3.4 Orchestration (Kubernetes)

Kubernetes is an **alternative serving surface** to Hugging Face Spaces. HF Spaces gives us one managed container. This there were no replicas, no rolling updates, no autoscaling except for the train, register, and deploy pipeline through GitHub Actions. This `k8s/` directory lets us run the same image anywhere where we can deploy a cluster:

- **`deployment.yaml`**: defines 2 replicas behind a `Deployment` controller. If a pod crashes, Kubernetes replaces it. Rolling updates are zero-downtime. `readinessProbe` keeps a pod out of the Service until it's actually healthy; `livenessProbe` restarts a pod that stops responding. Secrets management is done `NEW_RELIC_LICENSE_KEY` is injected from a Kubernetes Secret via `secretKeyRef` with `optional: true`, so pods start cleanly even without the secret. Resource scheduling is also done. `requests: 100m CPU / 256Mi RAM`, `limits: 1000m CPU / 1Gi RAM`.
- **`service.yaml`**: A `NodePort` on 30085 exposes port 8501.

An example *Secret* type YAML is added as well which can be added to inject New Relic or any other secrets.

3.5 CI/CD Pipeline (GitHub Actions)

`ci-cd.yml` defines four sequential jobs. Each job runs in a fresh Ubuntu runner; artifacts flow between jobs via `actions/upload-artifact` and `actions/download-artifact`.

Train

```
checkout -> setup-python 3.11 -> pip install -> pytest -> python -m src.train ->
upload model.pkl + metrics.json + mlruns/ as "model-bundle"
```

If `pytest` fails, the whole pipeline aborts before any artifact is produced.

Register

```
download "model-bundle" -> python -m src.register ->
cat registry/champion.json -> upload registry/champion.json as "registry"
```

This is where the champion/challenger decision happens.

Build-image

```
download both artifacts -> docker build ->
push ghcr.io/<owner>/insurance-app:latest
```

Uses the built-in `GITHUB_TOKEN` to authenticate against GHCR. The Docker image is built and pushed to GitHub Container Registry (GCHR).

Job 4 — deploy-hf

Three sub-steps:

1. **Sync HF Space secrets**: posts `NEW_RELIC_LICENSE_KEY` (as a secret) and `NEW_RELIC_APP_NAME` (as a variable) to the HF Spaces API. If the NR key isn't configured, this step emits a warning and continues.
2. **Build a clean orphan branch**: creates a `hf-deploy` branch from scratch, adds the code + model files with forceful `git add`.
3. **Push to HF Space**: pushes `hf-deploy:main` to HF Space using the HF token. HF detects the push, re-reads the Dockerfile, and rebuilds.

Required GitHub secrets:

Secret	Purpose
`HF_TOKEN`	Hugging Face write token for pushing to the Space.
`HF_USER`	Hugging Face username that owns the Space.
`NEW_RELIC_LICENSE_KEY`	Ingest license key (optional, pipeline continues gracefully without it).
`NEW_RELIC_APP_NAME`	App name shown in the New Relic UI (optional, defaults to `insurance-charge-predictor`).

3.6 Observability (New Relic)

New Relic is configured for observability. As this is a push based platform, all the system information is collected automatically. Custom metrics can be set up as well, which can be used for any ML-Specific observability concepts.

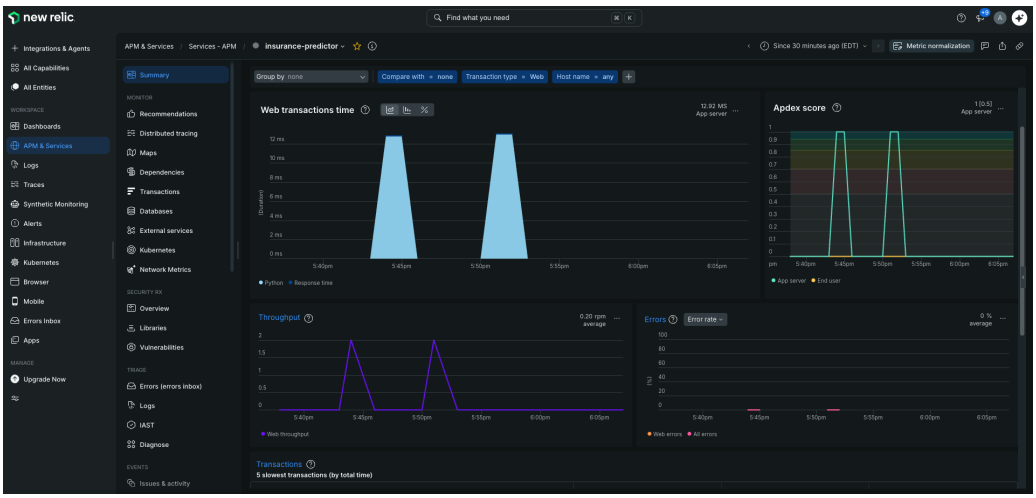

Figure 2: New Relic Service Summary

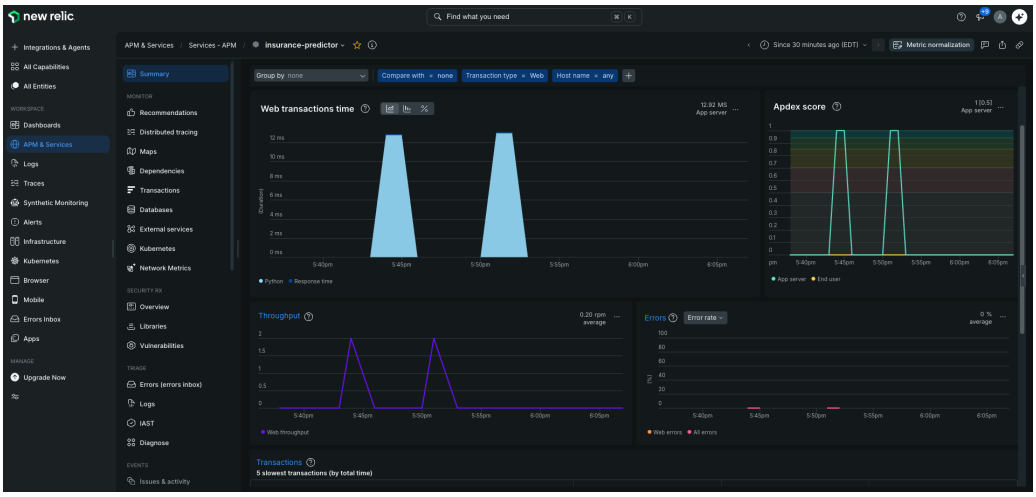

Figure 3: New Relic Transaction Details

4 Challenges Faced

- The main drawback of this project is that it is deployed in Hugging Face, which does not allow for a K8s orchestration. If resource was not an issue, the project can be easily managed using the file defined in `k8s/` folder.
- No custom metrics are set up, as the solution is not an API server. The current solution does not provide a process to detect any drift right now.
- DVC is configured within the system boundary, but it is not used effectively. The lack of a remote and the availability of only one dataset were the reasons for this.